

Special Application: Neural Style Transfer

What is Neural Style Transfer?

Neural style transfer


Content (C) Style (S)

Generated image (G)


Content (C) Style (S)

Generated image (G)

[Images generated by Justin Johnson]

Andrew Ng

[Save note](#)

Neural Style Transfer

Overview

Neural Style Transfer (NST) is a computer vision technique that generates a new image by combining:

- the **content** of one image
- the **style** of another image

The result is an image that preserves the original scene while adopting the artistic appearance of the style image.

Three Images

NST works with three images:

Symbol	Description
C	Content Image
S	Style Image
G	Generated Image

Content Image (C)
+
Style Image (S)
↓
Neural Style Transfer
↓
Generated Image (G)

Content Image (C)

The **content image** provides:

- Objects
- Scene layout
- Overall structure

Example:

- Stanford University campus
- Golden Gate Bridge

The generated image should preserve these high-level contents.

Style Image (S)

The **style image** provides the artistic appearance, including:

- Colors
- Brush strokes
- Texture
- Painting style

Examples:

- Van Gogh's *Starry Night*
- Pablo Picasso's paintings

The generated image imitates these artistic characteristics.

Generated Image (G)

The generated image combines:

- **Content** from **C**
- **Style** from **S**

Result:

Content Structure

+

Artistic Style

↓

Generated Artwork

Example 1

Content:
Stanford University Campus

+

Style:
Van Gogh - Starry Night

↓

Generated Image:
Stanford campus painted in Van Gogh's style

Example 2

Content:
Golden Gate Bridge

+

Style:
Picasso Painting

↓

Generated Image:
Golden Gate Bridge rendered in Picasso's style

Core Idea

Neural Style Transfer does **not** simply overlay one image on another.

Instead, it uses a **Convolutional Neural Network (CNN)** to extract different types of image features.

The generated image is optimized to simultaneously preserve:

- the content representation of the content image.
- the style representation of the style image.

Role of CNN Features

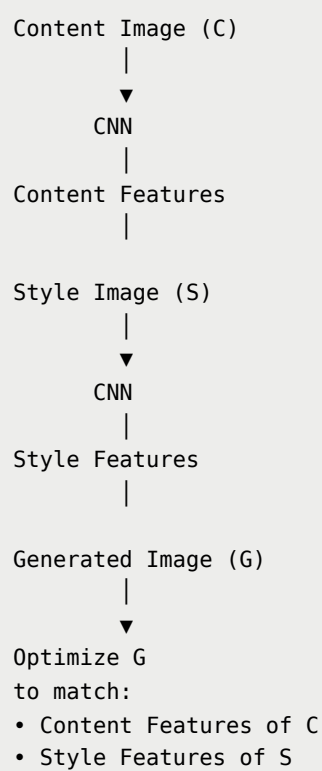
NST relies on features extracted from different CNN layers.

Different layers capture different information:

- **Shallow layers**
 - Low-level features
 - Edges
 - Colors
 - Textures
- **Deeper layers**
 - High-level semantic information
 - Object structure
 - Scene layout

These feature representations allow the network to separate **content** from **style**.

Neural Style Transfer Pipeline



Next Step

To implement Neural Style Transfer, it is important to understand **what different CNN layers learn**.

Specifically:

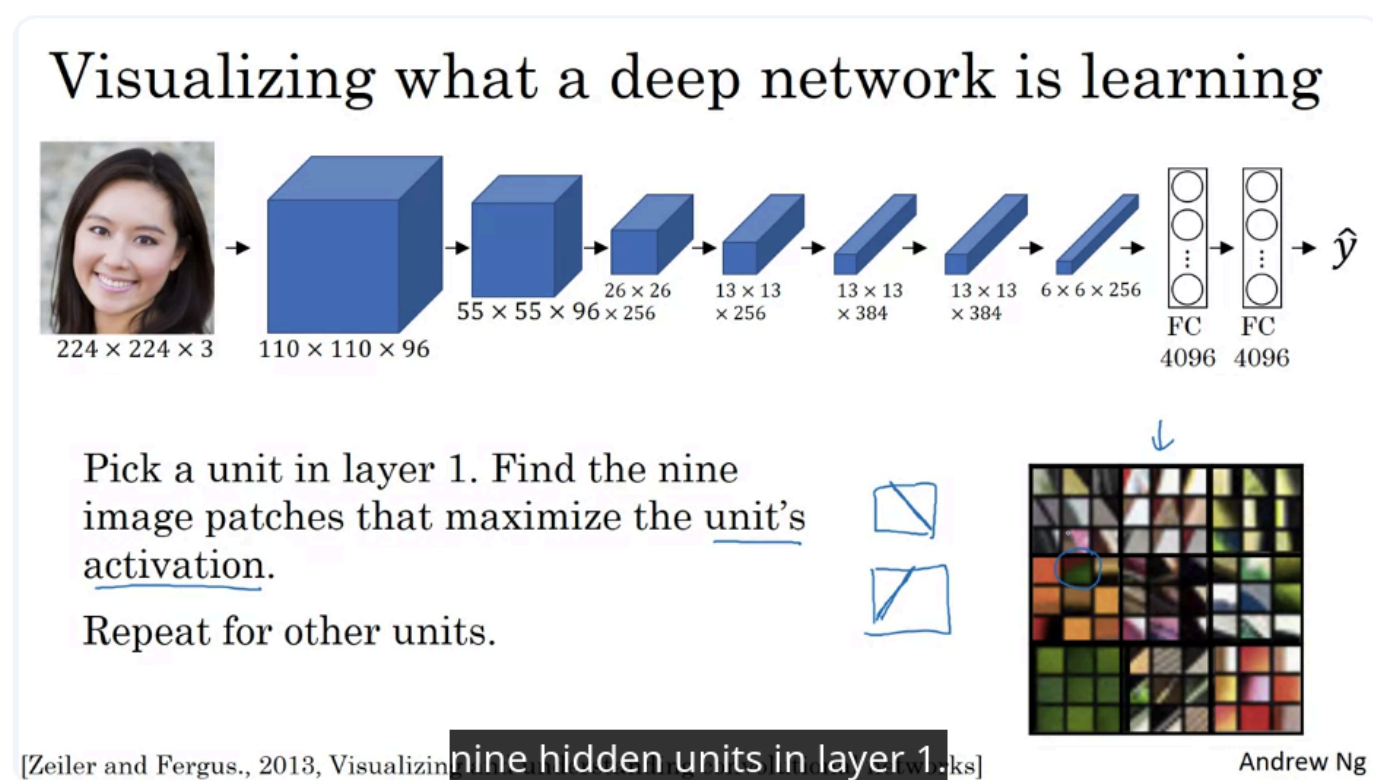
- Early layers capture low-level visual patterns.
- Deeper layers capture higher-level semantic representations.

These representations form the foundation of the Neural Style Transfer algorithm.

Key Takeaways

- **Neural Style Transfer (NST)** generates an image by combining the **content** of one image with the **style** of another.
- Three images are involved:
 - **Content Image (C)**
 - **Style Image (S)**
 - **Generated Image (G)**
- The generated image preserves the scene structure from the content image while adopting the artistic appearance of the style image.
- CNN feature representations enable the separation of **content** and **style**.
- Different CNN layers capture different levels of visual information, which are essential for implementing Neural Style Transfer.

What are deep ConvNets learning?



What are deep ConvNets learning?

[Save note](#)

Visualizing What Deep ConvNets Learn

Goal

Understand what hidden units in different layers of a ConvNet are learning by visualizing the input images that maximize their activations. This intuition is useful for understanding **Neural Style Transfer**.

Visualizing Hidden Units

For a hidden unit:

1. Pass all training images through the network.
2. Find the images (or image patches) that produce the highest activation.
3. Visualize these image patches.

For **Layer 1**:

- Each hidden unit has a **small receptive field**.

- Visualize only the image patch seen by that unit.
- Typically, display the top 9 image patches with the highest activation.

What Layer 1 Learns

Different hidden units respond to different simple features, such as:

- Edges
- Lines
- Vertical light edges
- Specific color patterns (e.g., green, orange, brownish-orange)

These visualizations show that early layers detect **low-level features**.

Deeper Layers

For hidden units in deeper layers:

- The **receptive field becomes larger**.
- Each unit can observe a larger region of the input image.
- In later layers, nearly every pixel can potentially influence a hidden unit's activation.

Although the visualized image patches are displayed at the same size, deeper-layer units actually see much larger portions of the image.

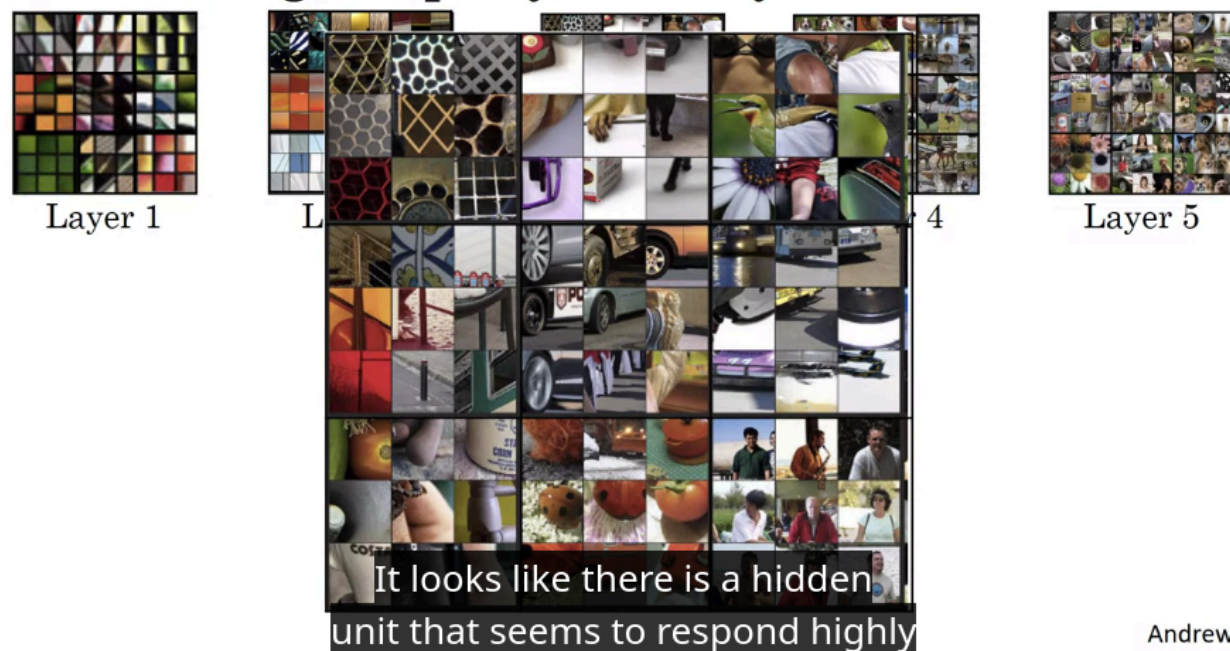
Reference

Examples are taken from the paper:

- *Visualizing and Understanding Convolutional Networks* by Matthew Zeiler and Rob Fergus.

The paper also introduces more advanced visualization methods beyond the simple activation-maximization approach discussed here.

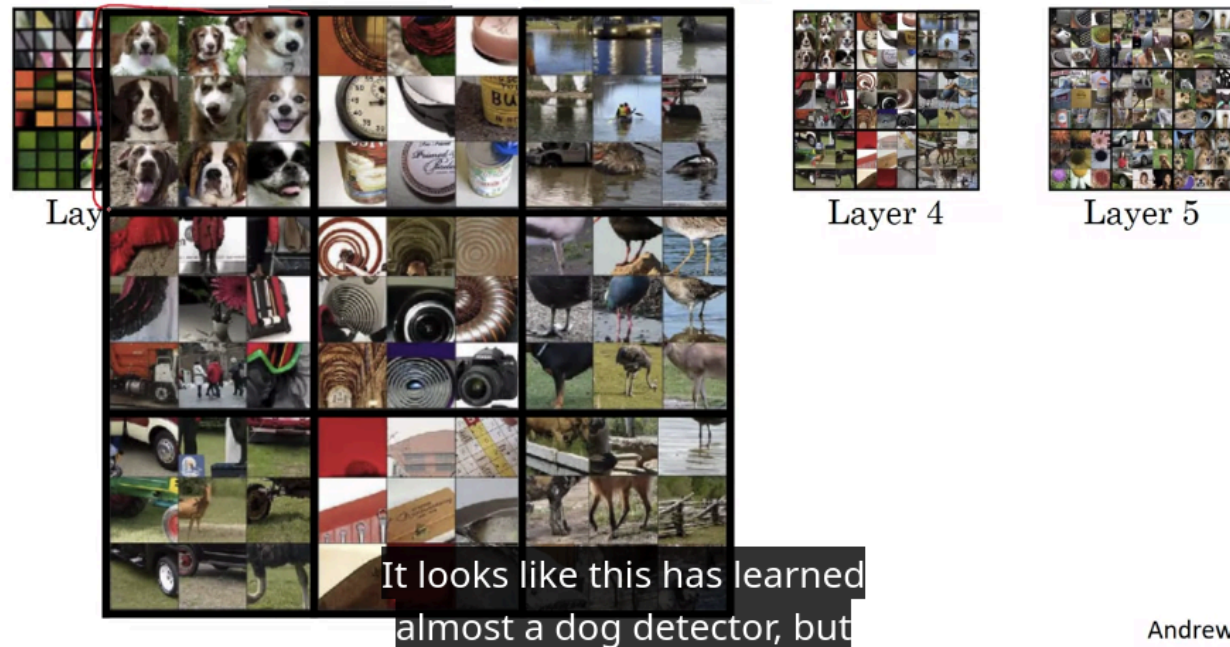
Visualizing deep layers: Layer 3



What are deep ConvNets learning?

[Save note](#)

Visualizing deep layers: Layer 4



Andrew Ng

What are deep ConvNets learning?

[Save note](#)

Visualizing Features in Deeper ConvNet Layers

Layer 2

The same visualization procedure is applied:

- For each hidden unit, find the 9 image patches that maximize its activation.
- Each group of 9 patches represents one hidden unit.

Compared to Layer 1, Layer 2 detects **more complex patterns**, such as:

- Vertical textures with multiple lines
- Rounded shapes
- Thin vertical lines

The network begins combining simple edges into more meaningful structures.

Layer 3

Layer 3 learns even higher-level features, including:

- Rounded object parts
- Textures (e.g., honeycomb or square-like patterns)
- Patterns commonly found in cars, dogs, and even people

Some learned features are difficult to interpret manually, but they clearly represent more complex visual concepts.

Layer 4

Features become increasingly object-specific:

- Dog-like shapes
- Water regions
- Bird legs

Hidden units start acting like detectors for particular object parts or object categories.

Layer 5

The deepest layer learns highly semantic features, including:

- Dogs (with greater variation than Layer 4)
- Keyboard-like textures
- Text

- Flowers

These neurons respond to complex objects rather than simple visual patterns.

Feature Hierarchy in ConvNets

As depth increases, learned features become progressively more abstract:

Layer	Learned Features
Layer 1	Edges, lines, colors
Layer 2	Textures and simple shapes
Layer 3	Object parts and complex textures
Layer 4	Specific object parts and categories
Layer 5	High-level semantic objects

Key Takeaway

ConvNets learn a hierarchical representation of images:

- **Early layers** detect simple visual features.
- **Middle layers** combine them into textures and object parts.
- **Deep layers** recognize complex objects and semantic concepts.

This hierarchical understanding provides the intuition needed for implementing **Neural Style Transfer**.

Cost Function

Neural Style Transfer: Cost Function

Problem formulation

- Given a content image C
- Given a style image S
- Goal: Generate a new image G



C

+

S

→

G

(content image)

(style image)

(generated image)

Cost function

- Content cost $J_{\text{content}}(C, G)$: measures how similar the contents of G are to the content of C .
- Style cost $J_{\text{style}}(S, G)$: measures how similar the style of G is to the style of S .
- α, β are hyperparameters controlling the relative weighting of the two costs.

$$J(G) = \alpha J_{\text{content}}(C, G) + \beta J_{\text{style}}(S, G)$$

Two hyperparameters are used (as in the original paper) to specify the relative weighting between content and style.

How the algorithm runs

1. Randomly initialize the generated image G (e.g., with random noise).
2. Compute the cost $J(G) = \alpha J_{\text{content}}(C, G) + \beta J_{\text{style}}(S, G)$.
3. Use gradient descent to update G and minimize $J(G)$.
4. Repeat until convergence (or for a fixed number of iterations).
5. The final G is the generated image.

Reference

Leon A. Gatys, Alexander S. Ecker, Matthias Bethge.
 "A Neural Algorithm of Artistic Style."
 arXiv: 1508.06576, 2015.

Neural Style Transfer Cost Function

Goal

Generate a new image **G** by combining:

- The **content** of image **C**
- The **style** of image **S**

This is achieved by defining a cost function and minimizing it using **gradient descent**.

Cost Function

The total cost is:

$$J(G) = \alpha J_{\text{content}}(C, G) + \beta J_{\text{style}}(S, G)$$

where:

- **Content Cost** $J_{\text{content}}(C, G)$
 - Measures how similar the content of **G** is to the content of **C**.
- **Style Cost** $J_{\text{style}}(S, G)$
 - Measures how similar the style of **G** is to the style of **S**.
- **α and β**
 - Hyperparameters controlling the relative importance of content and style.
 - Although one parameter could specify the balance, the original paper uses two, and this convention is followed.

Optimization Process

1. Define the total cost function **$J(G)$** .
2. Use **gradient descent** to minimize **$J(G)$** .
3. The optimized image becomes the generated image **G**.

Reference

The Neural Style Transfer algorithm presented here is based on the work of:

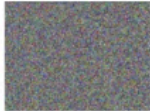
- Leon Gatys
- Alexander Ecker
- Matthias Bethge

The original paper is recommended for further reading.

Find the generated image G

1. Initiate G randomly
 $G: 100 \times 100 \times 3$
 $\uparrow$
RGB
2. Use gradient descent to minimize $J(G)$
$$G := G - \frac{\partial}{\partial G} J(G)$$





[Gatys et al., 2015. A neural algorithm of artistic style] Andrew Ng

your content image rendered in the style of your style image.

Cost Function [Save note](#)

Neural Style Transfer Optimization

Initialization

- Initialize the generated image **G** randomly.
- **G** can have any desired dimensions (e.g., `100 × 100 × 3` or `500 × 500 × 3`).

Optimization with Gradient Descent

After defining the total cost function $J(G)$, optimize **G** using gradient descent:

$$G := G - \alpha \frac{\partial J(G)}{\partial G}$$

- The optimization updates the **pixel values** of **G**.
- The generated image itself is the variable being optimized.

Generation Process

1. Start with:
 - Content image **C**
 - Style image **S**
2. Randomly initialize **G** as white noise.
3. Repeatedly update **G** using gradient descent.
4. As the cost decreases, **G** gradually:
 - Preserves the content of **C**
 - Adopts the style of **S**

Algorithm Summary

- Initialize **G** randomly.
- Define the total cost function $J(G)$.
- Minimize $J(G)$ by updating the pixels of **G** with gradient descent.
- The final optimized image combines the content of **C** with the style of **S**.

Content Cost Function

C: Content Image

G: Generated Image

Content cost function

$$J(G) = \alpha J_{\text{content}}(C, G) + \beta J_{\text{style}}(S, G)$$

- Say you use hidden layer l to compute content cost.
- Use pre-trained ConvNet. (E.g., VGG network)
- Let $a^{[l](C)}$ and $a^{[l](G)}$ be the activation of layer l on the images
- If $a^{[l](C)}$ and $a^{[l](G)}$ are similar, both images have similar content

$$J_{\text{content}}(C, G) = \frac{1}{2} \|a^{[l](C)} - a^{[l](G)}\|^2$$

on using this notation as if both of these have been unrolled into vectors,

Content Cost Function

Purpose

The content cost measures how similar the generated image **G** is to the content image **C**.

The total Neural Style Transfer cost has two parts:

$$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G)$$

This section focuses on defining:

$$J_{content}(C, G)$$

Choosing a Layer

Use a hidden layer l of a pre-trained ConvNet, such as VGG, to compare the content of **C** and **G**.

The choice of layer matters:

- If l is too shallow, the generated image will be forced to match the pixel values of **C** very closely.
- If l is too deep, the generated image only needs to contain high-level objects similar to **C**.
- In practice, l is chosen from the middle layers of the network.

Activations

Let:

$$a^{[l]}(C)$$

be the activation of layer l when the content image **C** is passed through the network.

Let:

$$a^{[l]}(G)$$

be the activation of layer l when the generated image **G** is passed through the network.

If these activations are similar, then **C** and **G** have similar content.

Content Cost

The content cost is defined as the squared difference between these activations:

$$J_{content}(C, G) = \left\| a^{[l]}(C) - a^{[l]}(G) \right\|^2$$

This means taking the element-wise difference between the activations, squaring them, and summing the results.

A normalization constant may be added, but it is not essential because its effect can be adjusted using the hyperparameter α .

Intuition

When minimizing the total cost $J(G)$, the content cost encourages **G** to produce hidden layer activations similar to **C**.

As a result, the generated image preserves the content of the content image.

Style Cost Function

Please note that in the next video at around 8:50 when Andrew wrote down the second formula of matrix, the second factor of the multiplication should be on index k' (k prime), not k (otherwise it does not calculate any kind of covariance). This is shown in black below.

$$G_{kk'}^{[l](G)} = \sum_{i=1}^{n_H} \sum_{j=1}^{n_W} a_{ijk}^{[l](G)} a_{ijk'}^{[l](G)}$$

So the formula should be:

$$G_{kk'}^{[l](G)} = \sum_{i=1}^{n_H} \sum_{j=1}^{n_W} a_{i,j,k}^{[l](G)} a_{i,j,k'}^{[l](G)}$$

Also at 11:08 the style cost function formula should be the squared difference:

$$(G_S - G_G)^2$$

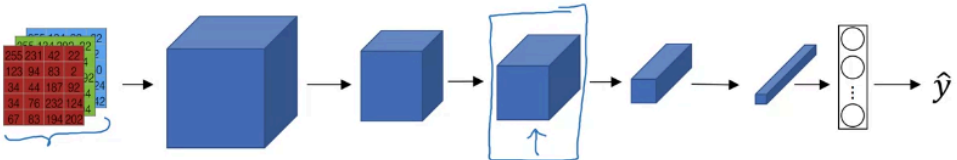
instead of just the difference:

$$(G_S - G_G)$$

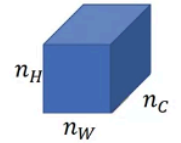
The style cost function should be:

$$J_{style}^{[l]}(S, G) = \frac{1}{(2n_H^{[l]}n_W^{[l]}n_C^{[l]})^2} \sum_k \sum_{k'} \left(G_{kk'}^{[l](S)} - G_{kk'}^{[l](G)} \right)^2$$

Meaning of the “style” of an image



Say you are using layer l 's activation to measure “style.”
Define style as correlation between activations across channels.



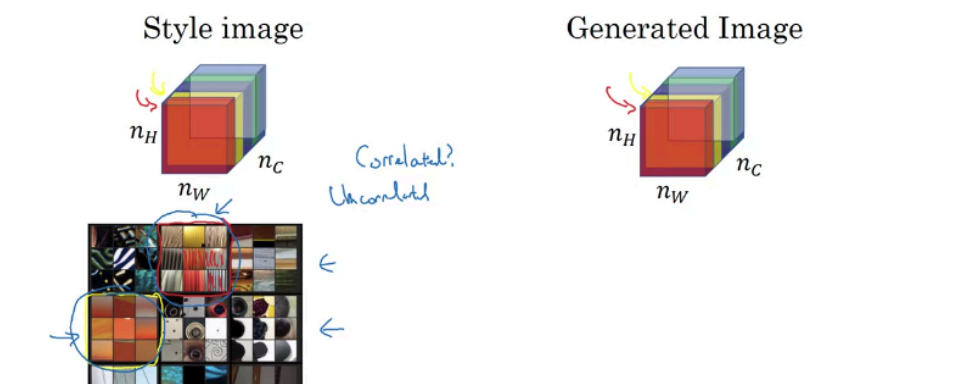
How correlated are the activations across different channels?

[Gatys et al., 2015. A neural algorithm of artistic style] **let's take this block of activations**

Andrew Ng

Style Cost Function [Save note](#)

Intuition about style of an image



of how similar is the style of the generated image to the style of the input style image.

[Gatys et al., 2015. A neural algorithm of artistic style] **style image.**

Andrew Ng

Style Cost Function [Save note](#)

Style Cost Intuition

Purpose

The style cost measures how similar the **style** of the generated image **G** is to the style image **S**.

Unlike content, style is represented by the **correlation between feature channels** in a hidden layer.

Feature Activations

Choose a hidden layer (l) of a pre-trained ConvNet.

The activation tensor has dimensions:

$$n_H \times n_W \times n_C$$

where:

- (n_H): height
- (n_W): width
- (n_C): number of feature channels

Channel Correlation

To represent style:

- Compare activations between different feature channels.
- For each pair of channels, measure how their activations are correlated across all spatial positions ($n_H \times n_W$).

The correlation captures how often two feature patterns appear together in the image.

Intuition

Suppose:

- Channel 1 (the red layer) detects a **vertical texture**.
- Channel 2 (the yellow layer) detects **orange-colored regions**.

Then:

- **High correlation**
 - Vertical textures frequently appear with orange regions.
- **Low correlation**
 - The two features occur independently.

Thus, channel correlations describe relationships between high-level visual patterns rather than individual objects.

Why Correlation Represents Style

The correlation between feature channels captures:

- Which texture patterns appear together.
- Which visual features co-occur across the image.

These relationships characterize the artistic style of an image.

Style Matching

For Neural Style Transfer:

- Compute channel correlations for the style image **S**.
- Compute the same correlations for the generated image **G**.
- The style cost encourages **G** to have channel correlations similar to **S**, resulting in a similar visual style.

Style Matrix

For a chosen layer l , let:

$$a_{i,j,k}^{[l]}$$

be the activation at:

- i : height index
- j : width index
- k : channel index

The **style matrix** is an $n_C \times n_C$ matrix:

$$G^{[l]}$$

Each element measures the correlation between channels k and k' :

$$G_{k,k'}^{[l]} = \sum_{i=1}^{n_H} \sum_{j=1}^{n_W} a_{i,j,k}^{[l]} a_{i,j,k'}^{[l]}$$

Interpretation

- Large $G_{k,k'}^{[l]}$: channels k and k' are highly correlated.
- Small $G_{k,k'}^{[l]}$: the channels are weakly correlated.

The style matrix captures how different feature channels co-occur throughout the image.

Style Cost for One Layer

Compute the style matrices for both:

- Style image: $G^{[l]}(S)$
- Generated image: $G^{[l]}(G)$

The style cost for layer l is the squared difference between the two style matrices:

$$J_{style}^{[l]}(S, G) = \left\| G^{[l]}(S) - G^{[l]}(G) \right\|_F^2$$

where $\| \cdot \|_F$ is the Frobenius norm.

A normalization constant may be included, but its effect is adjusted by the weighting parameter β .

Multi-Layer Style Cost

Better visual results are obtained by combining style costs from multiple layers:

$$J_{style}(S, G) = \sum_l \lambda^{[l]} J_{style}^{[l]}(S, G)$$

where:

- $\lambda^{[l]}$ controls the contribution of each layer.
- Early layers capture low-level features (e.g., edges).
- Deeper layers capture higher-level visual patterns.

Final Objective

The overall Neural Style Transfer objective is:

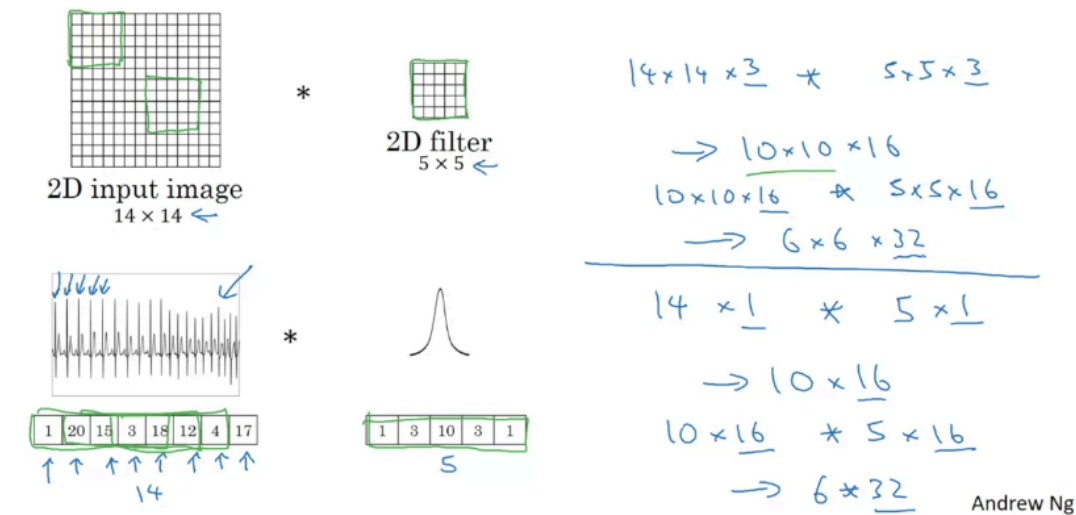
$$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G)$$

The generated image $\mathbf{G}$ is optimized by minimizing $J(G)$ using gradient descent (or another optimization algorithm). This produces an image that preserves the content of $\mathbf{C}$ while matching the style of $\mathbf{S}$.

🔍 ↶ ↷ ↵ ↶ ↷

1D and 3D Generalizations

Convolutions in 2D and 1D



1D Convolutions

Generalization of ConvNets

Although ConvNets are commonly used for **2D images**, the same ideas can be applied to **1D** and **3D** data.

From 2D to 1D Convolution

For 2D images:

- Input: 14×14
- Filter: 5×5
- Output: 10×10

With multiple channels:

- Input: $14 \times 14 \times 3$
- Filter: $5 \times 5 \times 3$
- 16 filters produce an output of:

$10 \times 10 \times 16$

For 1D data:

- Input: 14
- Filter: 5
- Output:

10

With multiple filters:

- Input: 14×1
- Filter: 5×1
- 16 filters produce:

10×16

Stacking 1D Convolution Layers

Example of a second convolution layer:

- Input: 10×16
- Filter: 5×16

- 32 filters produce:

$$6 \times 32$$

The number of filter channels must match the number of input channels.

Example: EKG Signal

An **EKG (Electrocardiogram)** is a 1D time-series signal that records electrical activity of the heart.

A 1D convolution applies the same feature detector across different time positions, allowing the network to detect patterns such as heartbeats throughout the signal.

ConvNets for Sequential Data

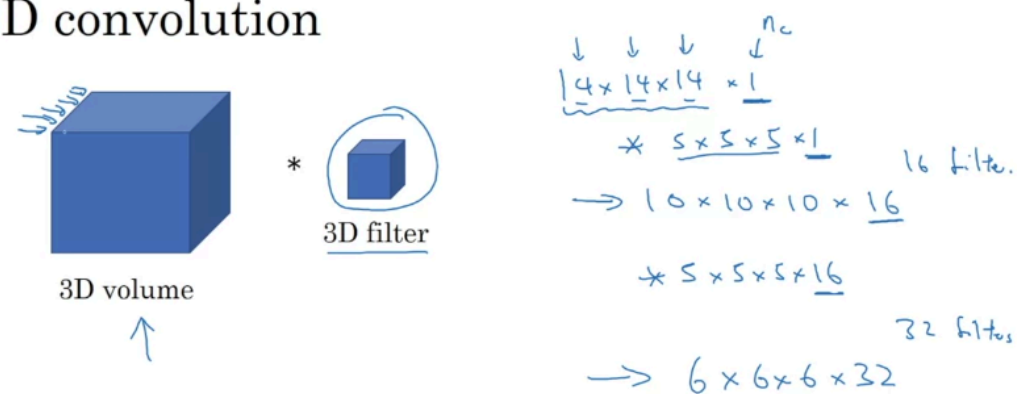
ConvNets can be applied to 1D sequential data.

However, for many sequence tasks, **Recurrent Neural Networks (RNNs)** and related models are also commonly used. Their advantages and trade-offs are discussed in the next course on sequence models.

3D Data



3D convolution



And you could use this to detect motion or people taking actions in movies.

Andrew Ng

3D Convolutions

3D Data

Unlike 1D sequences or 2D images, **3D data** consists of a volume with:

- Height
- Width
- Depth

A common example is a **CT (CAT) scan**, which represents the body as multiple slices along the depth dimension.

3D Convolution

Example:

- Input volume:

$14 \times 14 \times 14$

- Filter:

$5 \times 5 \times 5$

- Output:

$10 \times 10 \times 10$

With channels:

- Input:

$14 \times 14 \times 14 \times 1$

- Filter:

$5 \times 5 \times 5 \times 1$

- Using 16 filters produces:

$10 \times 10 \times 10 \times 16$

Stacking 3D Convolution Layers

Example of the next layer:

- Input:

$10 \times 10 \times 10 \times 16$

- Filter:

$5 \times 5 \times 5 \times 16$

- Using 32 filters produces:

$6 \times 6 \times 6 \times 32$

As in 2D ConvNets, the number of filter channels must match the number of input channels.

Applications

3D ConvNets can learn features directly from volumetric data, including:

- CT/CAT scans and other medical imaging.
- Video data, where the depth dimension represents time, enabling motion and action recognition.

Summary

The convolution operation generalizes naturally across different data types:

Data Type	Example	Filter
1D	Time-series, EKG	1D filter
2D	Images	2D filter
3D	CT scans, Videos	3D filter

The same principles of convolution, parameter sharing, and feature detection apply to 1D, 2D, and 3D data.